# Appendix – ARIMA Model

| Ticker | **ARIMA order** | **MAE (平均絕對誤差)** | **RMSE (均方根誤差)** | **MAPE (平均絕對百分比誤差 %)** | **$R^2$ (決定係數)** |
|---|---|---|---|---|---|
| 0386.HK | (0, 1, 0) | 0.3170 | 0.3678 | 7.17 | -0.0623 |
| 0388.HK | (0, 1, 0) | 27.2592 | 30.6604 | 7.01 | -3.6749 |
| 0005.HK | (3, 1, 0) | 11.8207 | 13.8846 | 7.81 | -0.0337 |
| ^HSI | (0, 1, 0) | 1400.7243 | 1661.9426 | 5.63 | -1.9371 |
| ^GSPC | (1, 1, 1) | 156.0880 | 222.4891 | 2.23 | 0.6512 |

```python
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pmdarima import auto_arima
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib
import os
import xlsxwriter

# 設定字型避免中文亂碼
matplotlib.rcParams['font.sans-serif'] = ['SimHei']
matplotlib.rcParams['axes.unicode_minus'] = False

tickers = ["0386.HK", "0388.HK", "0005.HK", "^HSI", "^GSPC"]
os.makedirs("results", exist_ok=True)

all_results = []
plot_files = []

for ticker in tickers:
    # 1. 下載股價
    df = yf.download(ticker, start="2023-01-01", end="2026-09-28", progress=False)

    # ★★★ 關鍵修正：強制把 Close 變成一維 Series ★★★
    close = df["Close"]
    if isinstance(close, pd.DataFrame):
        close = close.iloc[:, 0]          # 取出第一欄
    close = close.squeeze().dropna().reset_index(drop=True)

    # 存成原始收市價 CSV
    filename = f"results/Close_price_{ticker.replace('^', '')}_raw.csv"
    df.to_csv(filename, index=True, encoding="utf-8-sig")

    # 2. 分割訓練集與測試集
    train = close.iloc[:746]
    test  = close.iloc[746:]

    # 3. auto_arima
    model = auto_arima(
        train,
        start_p=0, start_q=0,
        max_p=5, max_q=5,
        d=None,
        seasonal=False,
```

```
        stepwise=True,
        suppress_warnings=True,
        error_action='ignore'
    )
    best_order = model.order

    # 4. 預測
    n_periods = len(test)
    forecast = model.predict(n_periods=n_periods)
    forecast = pd.Series(forecast, index=test.index)

    # 5. 計算評估指標
    mae  = mean_absolute_error(test, forecast)
    rmse = np.sqrt(mean_squared_error(test, forecast))
    r2   = r2_score(test, forecast)

    # ★★★ MAPE 計算（確保是純數值）★★★
    mape = np.mean(np.abs((test.values - forecast.values) / (test.values + 1e-8))) * 100

    # 收集結果
    all_results.append({
        "Ticker": ticker,
        "ARIMA order": str(best_order),
        "MAE (平均絕對誤差)": mae,
        "RMSE (均方根誤差)": rmse,
        "MAPE (平均絕對百分比誤差 %)": mape,   # 現在一定是 float
        "R² (決定係數)": r2
    })

    # 6. 繪圖
    plt.figure(figsize=(14, 8))
    plt.plot(train.index, train, label="訓練集（實際）", color="blue")
    plt.plot(test.index, test, label="測試集（實際）", color="green")
    plt.plot(test.index, forecast, label=f"ARIMA{best_order} 預測", color="red", linestyle="--")
    plt.axvline(x=len(train)-1, color="gray", linestyle=":", label="訓練/測試分界")



    # 設定垂直格線每 124.33333 一個
    ax = plt.gca()
    ax.xaxis.set_major_locator(plt.MultipleLocator(124.33333))   # 垂直格線間隔 124.33333
    ax.grid(True, axis='x', which='major', alpha=0.3)        # 只顯示垂直格線
    ax.grid(True, axis='y', alpha=0.3)                       # 水平格線保持原樣（可選）



    plt.title(f"{ticker} ARIMA{best_order} 預測 vs 實際收盤價")
```

```python
    plt.title(f"{ticker} ARIMA{best_order} 預測 vs 實際收市價\n"
              f"MAE={mae:.2f}, RMSE={rmse:.2f}, MAPE={mape:.2f}%, R²={r2:.2f}", fontsize=14)
    plt.xlabel("時間 (索引)")
    plt.ylabel("收市價")
    plt.legend()
    plt.grid(True, alpha=0.3)

    plot_filename = f"results/arima_plot_{ticker.replace('^', '')}.png"
    plt.savefig(plot_filename, dpi=300, bbox_inches="tight")
    plt.close()
    plot_files.append((ticker, plot_filename))

# 存評估結果
df_all = pd.DataFrame(all_results)
df_all.to_csv("results/arima_evaluation_results_all.csv", index=False, encoding="utf-8-sig")

# Excel（圖 + Summary）
excel_filename = "results/arima_plots_all.xlsx"
workbook = xlsxwriter.Workbook(excel_filename)

for ticker, plot_file in plot_files:
    sheet_name = ticker.replace("^", "")
    worksheet = workbook.add_worksheet(sheet_name)
    worksheet.insert_image("B2", plot_file)

# Summary sheet
summary_ws = workbook.add_worksheet("Summary")
for col_num, col_name in enumerate(df_all.columns):
    summary_ws.write(0, col_num, col_name)

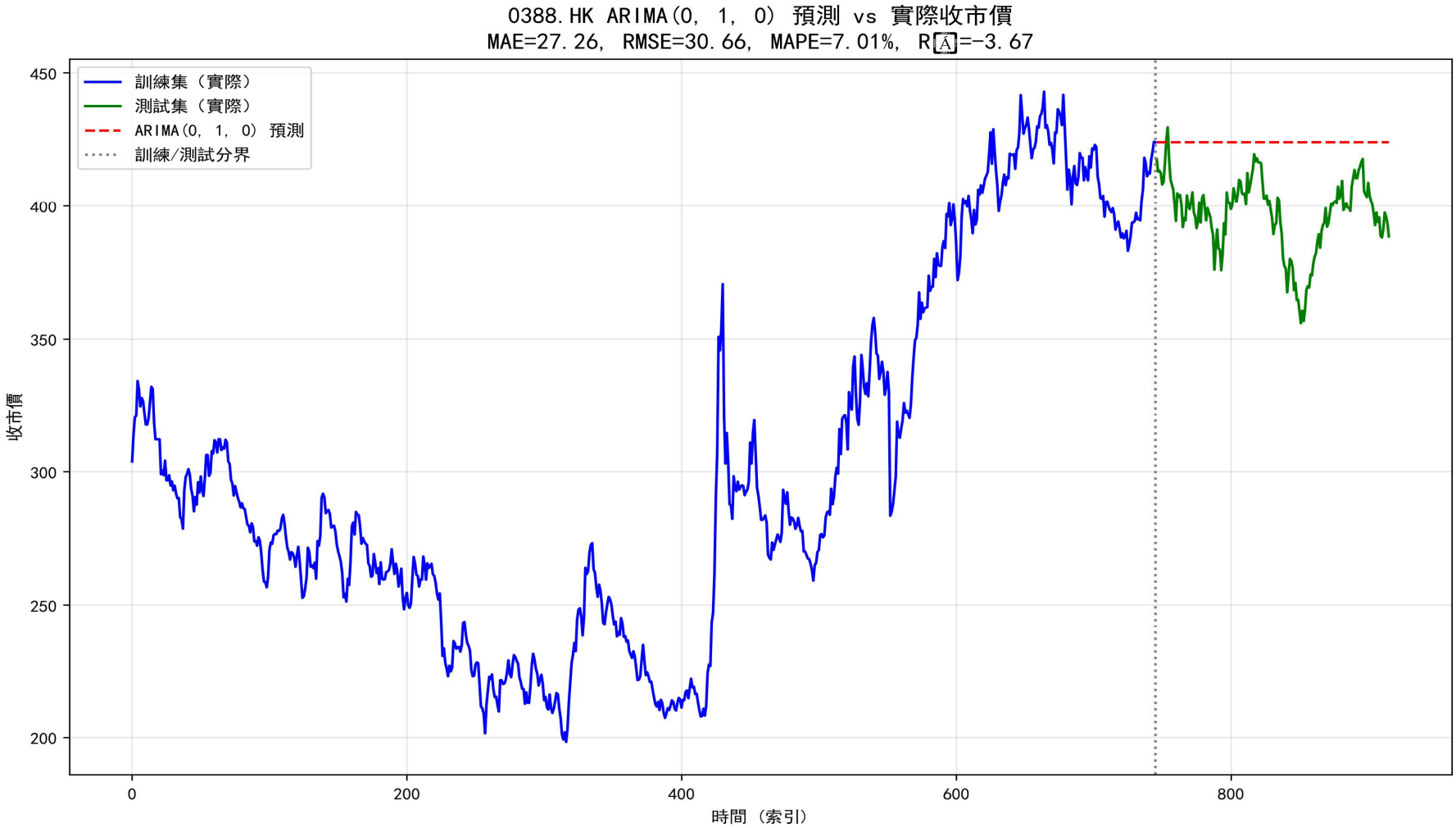
for row_num, row in enumerate(df_all.itertuples(index=False), start=1):
    for col_num, value in enumerate(row):
        # 確保寫入的是純數值或字串
        if isinstance(value, (float, np.floating)):
            summary_ws.write(row_num, col_num, float(value))
        else:
            summary_ws.write(row_num, col_num, str(value))

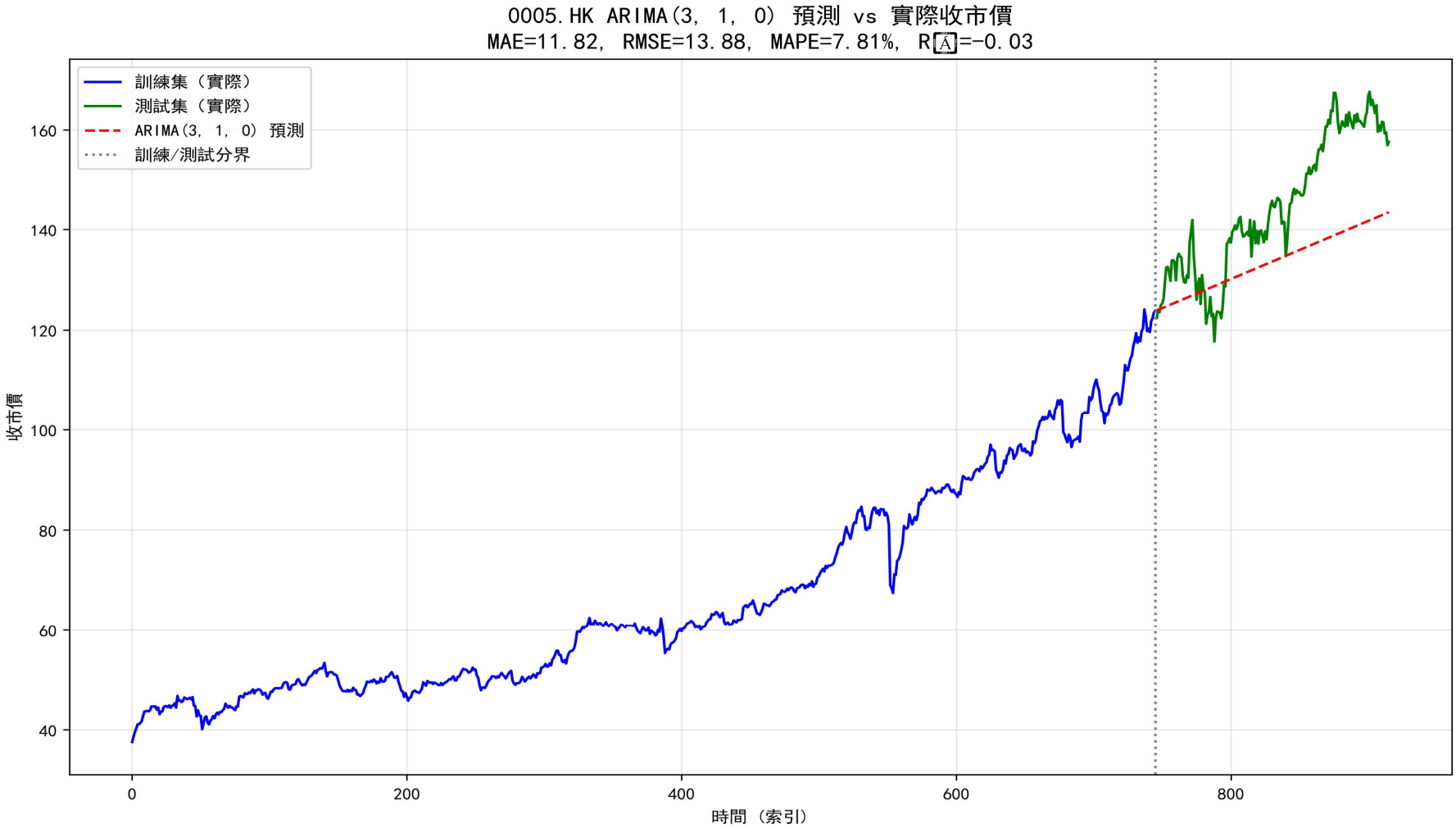
workbook.close()

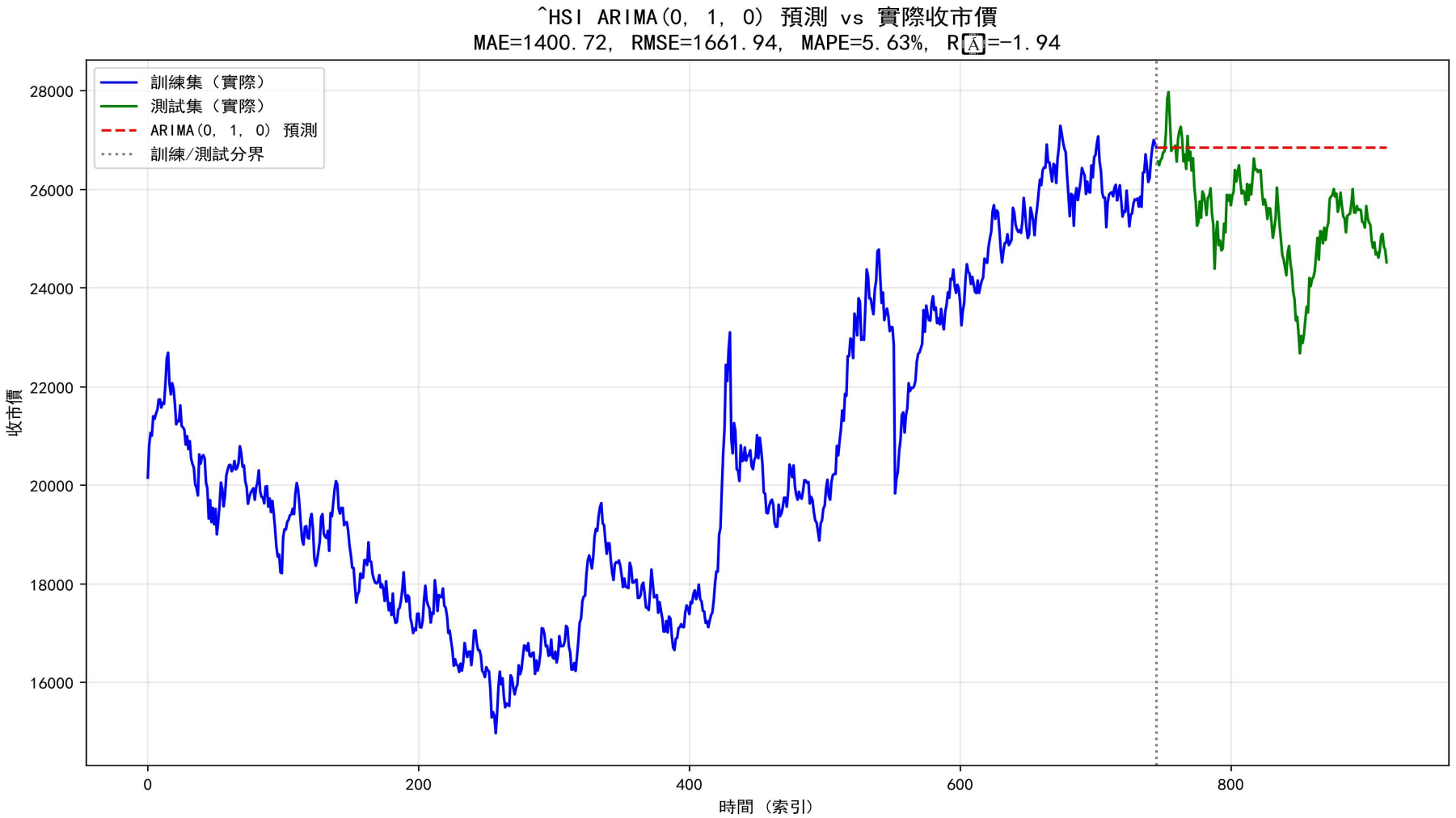
print("完成! 請檢查 results/arima_evaluation_results_all.csv 與 Excel 的 Summary 工作表")
print(df_all)
```

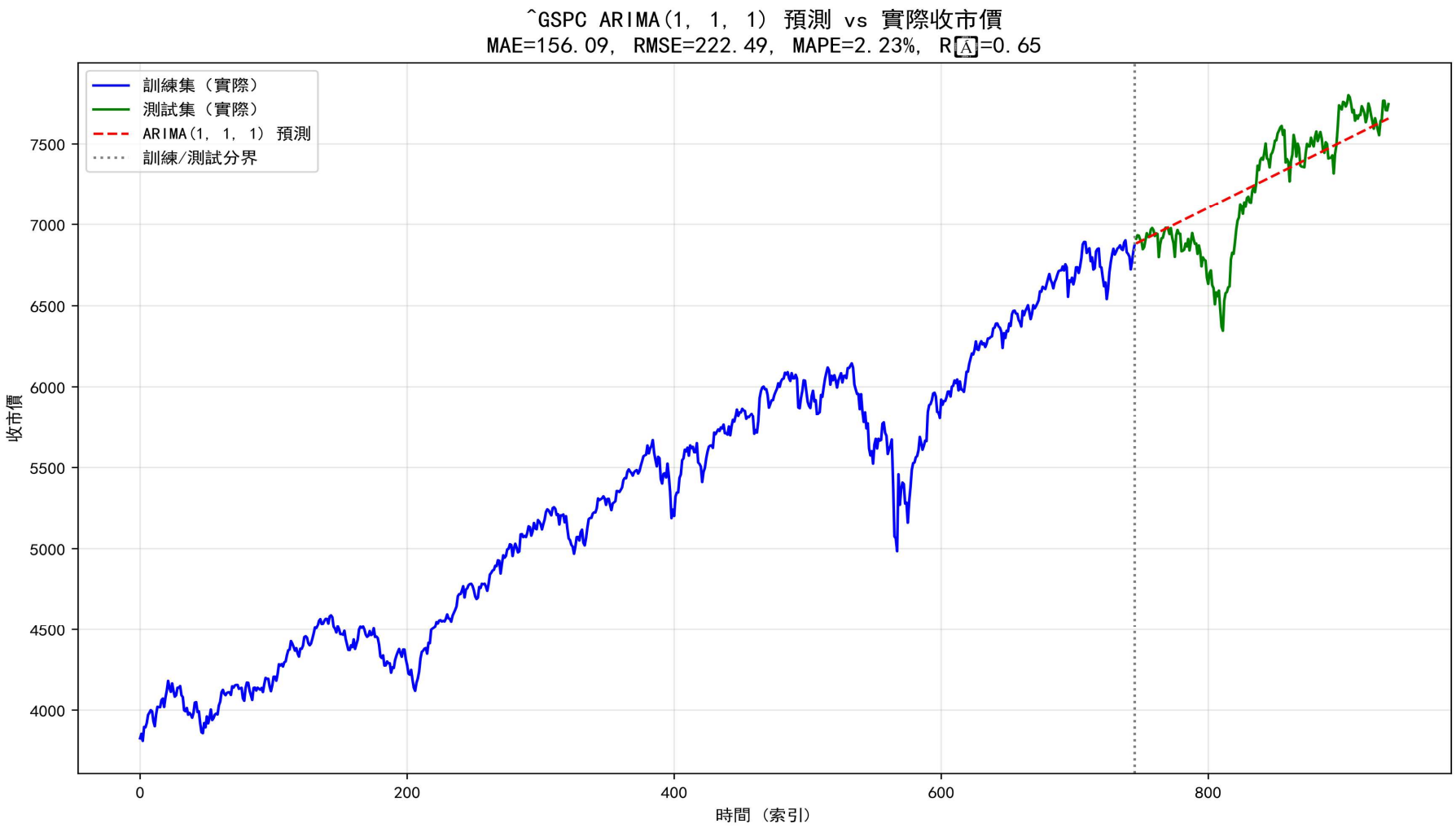
Python


0386.HK ARIMA(0, 1, 0) 預測 vs 實際收市價
MAE=0.32, RMSE=0.37, MAPE=7.17%, R[A]=-0.06
訓練集（實際）
測試集（實際）
ARIMA(0, 1, 0) 預測
訓練/測試分界
收市價
時間（索引）
5.0
4.5
4.0
3.5
3.0
0
200
400
600
800


0388.HK ARIMA(0, 1, 0) 預測 vs 實際收市價
MAE=27.26, RMSE=30.66, MAPE=7.01%, R²=-3.67
訓練集（實際）
測試集（實際）
ARIMA(0, 1, 0) 預測
訓練/測試分界
收市價
時間（索引）
450
400
350
300
250
200
0
200
400
600
800


0005.HK ARIMA(3, 1, 0) 預測 vs 實際收市價
MAE=11.82, RMSE=13.88, MAPE=7.81%, R□=-0.03
訓練集（實際）
測試集（實際）
ARIMA(3, 1, 0) 預測
訓練/測試分界
收市價
時間（索引）
40
60
80
100
120
140
160
0
200
400
600
800


^HSI ARIMA(0, 1, 0) 預測 vs 實際收市價
MAE=1400.72, RMSE=1661.94, MAPE=5.63%, R□=-1.94
訓練集（實際）
測試集（實際）
ARIMA(0, 1, 0) 預測
訓練/測試分界
收市價
時間（索引）
28000
26000
24000
22000
20000
18000
16000
0
200
400
600
800


^GSPC ARIMA(1, 1, 1) 預測 vs 實際收市價
MAE=156.09, RMSE=222.49, MAPE=2.23%, R□=0.65
訓練集（實際）
測試集（實際）
ARIMA(1, 1, 1) 預測
訓練/測試分界
收市價
時間（索引）
7500
7000
6500
6000
5500
5000
4500
4000
0
200
400
600
800